React Interview Prep

Topic 5 — useEffect: Complete Guide

Analogy · 3 Forms · Cleanup · Fetch · Stale Closure · AbortController · 8 Misconceptions

■ Covers	All examples from conversation — nothing skipped
■ Series	React Interview Prep — Session 5

by Akshay Chavhan • React Interview Series

1. Like You're 5 — Moving Into a House

Imagine you just moved into a new house (component renders):

Turn on lights when you arrive → **Side Effect (useEffect runs)** → **Turn off when you leave**

useEffect is React's way of saying: After you finish painting the UI — go do this extra work.

A side effect is anything that reaches OUTSIDE the component:

- Fetching API data
- Setting up a timer / interval
- Subscribing to events
- Manually touching the DOM
- Updating document.title

> ■ ***useEffect runs AFTER the component renders and the browser has painted — not during render.***

2. The 3 Forms of useEffect

```
// Form 1 — runs after EVERY render (no array)
useEffect(() => { console.log('runs every render'); });

// Form 2 — runs ONLY on first render (mount)
useEffect(() => { console.log('mount only'); }, []);

// Form 3 — runs on mount + when dependency changes
useEffect(() => { console.log('count changed'); }, [count]);
```

Form	Dependency Array	When It Runs
Form 1	No array	After every single render
Form 2	Empty [ ]	Only on mount (first render)
Form 3	[a, b]	On mount + when a or b changes

3. Cleanup Function — When & Why

The function you return from useEffect. React runs it in TWO moments:

1. Before the next effect runs (when dependency changes)
2. When the component unmounts (removed from DOM)

```
useEffect(() => {
const timer = setInterval(() => console.log('tick'), 1000);

return () => {
clearInterval(timer); // cleanup — stops the timer
};
}, []);

// userId: 1 → 2 (cleanup sequence)
// ■ effect runs for userId: 1
// ■ cleanup runs for userId: 1 ← before next effect, NOT just on unmount
// ■ effect runs for userId: 2
```

■ *Cleanup runs before the NEXT effect AND on unmount. Not just on unmount!*

4. Real World — Data Fetching

```
function UserProfile({ userId }) {
const [user, setUser] = useState(null);
const [loading, setLoading] = useState(true);
useEffect(() => {
setLoading(true);
fetch(`/api/users/${userId}`)
.then(res => res.json())
.then(data => {
setUser(data);
setLoading(false);
});
}, [userId]); // re-fetch when userId changes
if (loading) return <p>Loading...</p>;
return <h1>{user?.name}</h1>;
}
```

5. AbortController — Fixing Race Conditions

When a dependency changes mid-flight, the old fetch must be cancelled or stale data overwrites fresh data.

```
useEffect(() => {
const controller = new AbortController();
fetch(`/api/users/${userId}`, { signal: controller.signal })
.then(res => res.json())
.then(data => setUser(data))
.catch(err => {
if (err.name === 'AbortError') return; // cancelled — ignore
console.error(err);
});
return () => controller.abort(); // cancel if userId changes mid-flight
}, [userId]);
```

> ■ ***Without abort: User 1 fetch slower than User 2 → User 1 data overwrites User 2 = wrong data shown.***

6. Stale Closure Problem & Fix

■ Wrong

```
useEffect(() => {
const timer = setInterval(() => {
console.log(count);
// ■ always prints 0 — stale!
// captured count=0 at mount
}, 1000);
return () => clearInterval(timer);
}, []); // [] = never re-runs
```

■ Correct

```
// Fix 1 — add count to deps
useEffect(() => {
const timer = setInterval(() => {
console.log(count); // fresh ■
}, 1000);
return () => clearInterval(timer);
}, [count]);

// Fix 2 — functional update
setCount(prev => prev + 1); // ■
```

7. 8 Common Misconceptions About useEffect

Misconception 1 — Runs during render

```
render runs ← happens FIRST (during render)
effect runs ← happens AFTER browser paints — NEVER during render
```

> ■ ***Truth: useEffect runs AFTER render is committed to DOM and browser has painted.***

Misconception 2 — Empty [] means run once forever

■ Wrong

```
useEffect(() => {
console.log('once?');
}, []);
// Runs TWICE in React 18 Strict Mode dev!
```

■ Correct

```
// [] = no dependencies — not run once
// Production: runs once ■
// Strict Mode dev: runs twice
// intentional — exposes cleanup bugs
```

> ■ ***Truth: [] = no dependencies. Strict Mode double-invokes in dev to catch cleanup bugs.***

Misconception 3 — useEffect can be async directly

■ Wrong

```
// async returns a Promise
// Promise is not a cleanup function
useEffect(async () => {
const data = await fetch('/api');
setUser(data);
}, []); // React warns ■
```

■ Correct

```
// Define async inside, call it
useEffect(() => {
async function load() {
const res = await fetch('/api');
const data = await res.json();
setUser(data);
}
load(); // call immediately ■
}, []);
```

Misconception 4 — Same as componentDidMount exactly

■ Wrong

```
// componentDidMount: BEFORE paint (sync)
// useEffect([]): AFTER paint (async)
// May cause flicker:
useEffect(() => {
const h = ref.current.offsetHeight;
setHeight(h); // flicker possible ■
}, []);
```

■ Correct

```
// useLayoutEffect = true equivalent
// Runs synchronously before paint
useLayoutEffect(() => {
const h = ref.current.offsetHeight;
setHeight(h); // no flicker ■
}, []);
```

Misconception 5 — Objects/Arrays in deps are fine

■ Wrong

```
// New object reference every render!
const config = { theme: 'dark' };
useEffect(() => {
doSomething(config);
}, [config]); // ■ infinite loop!
```

■ Correct

```
// Fix 1: use primitive
useEffect(() => {
doSomething(config);
}, [config.theme]); // string ■

// Fix 2: useMemo
const config = useMemo(
() => ({ theme: 'dark' }), []
);
```

> ***■ Truth: React compares deps by reference. New object each render = always changed = infinite loop.***

Misconception 6 — Cleanup only runs on unmount

Cleanup runs in TWO moments — before next effect AND on unmount.

```
// userId: 1 → 2
// ■ effect for userId: 1
// ■ cleanup for userId: 1 ← before next effect
// ■ effect for userId: 2
```

Misconception 7 — useEffect is right for all side effects

■ Wrong

```
// Extra render + unnecessary effect
useEffect(() => {
setDisplayName(userId.toUpperCase(
));
}, [userId]);

return <p>{displayName}</p>;
```

■ Correct

```
// Just compute during render!
const displayName =
userId.toUpperCase();

return <p>{displayName}</p>;

// useEffect = external world only
■
```

Misconception 8 — Deps are optional, add what you want

■ Wrong

```
// Lying to React — name missing
useEffect(() => {
console.log(name, count);
}, [count]); // name is stale! ■
```

■ Correct

```
// Honest deps — always fresh
useEffect(() => {
console.log(name, count);
}, [name, count]); // ■

// ESLint exhaustive-deps enforces this
```

> ***■ Truth: Deps array is a contract. Every value used inside must be listed. Missing = stale closure bugs.***

8. All 8 Misconceptions — Quick Reference

Misconception	Truth
Runs during render	Runs AFTER browser paints
[] = run once forever	[] = no deps (twice in Strict Mode dev)
Can be async directly	Wrap async inside and call it
Same as componentDidMount	Use useLayoutEffect for DOM measurements
Object in deps is fine	New reference each render = infinite loop
Cleanup only on unmount	Also before next effect on dep change
Good for all side effects	Only for external world — not data transforms
Deps are optional	Contract with React — missing = stale bugs

9. useEffect vs useLayoutEffect

Hook	When it runs	Use for
useEffect	AFTER browser paints (async)	Data fetching, subscriptions, logging
useLayoutEffect	BEFORE browser paints (sync)	DOM measurements, preventing visual flicker

■ ***99% of the time use useEffect. Only reach for useLayoutEffect when you see a visual flicker.***

10. Interview Q&A;

Q Beginner **Q1. What is useEffect used for?**

➜ useEffect is for side effects — things that reach outside React like fetching data, setting up timers, subscribing to events, or updating document.title. It runs after the component renders so it never blocks the UI.

Q Beginner **Q2. What are the 3 forms of useEffect?**

➜ No dependency array → runs after every render. Empty array [] → runs only on mount. Array with values [a, b] → runs on mount and whenever a or b changes.

Q Intermediate **Q3. What is the cleanup function and when does it run?**

➜ It's a function returned from useEffect. React runs it before the next effect executes (when a dependency changes) AND when the component unmounts. Used to clear timers, cancel subscriptions, or abort fetch requests.

Q Intermediate

Q4. What is a stale closure in useEffect?

➜ When an effect captures a variable at creation time and never updates — like reading count inside setInterval with an empty dep array. Fix: add the variable to dependencies, or use functional state updates.

Q Intermediate

Q5. Why can't useEffect be async directly?

➜ An async function returns a Promise. useEffect expects its return value to be either undefined or a cleanup function. A Promise is neither — React will warn you and the cleanup won't work.

Q Expert

Q6. What is the difference between useEffect and useLayoutEffect?

➜ useEffect runs asynchronously after the browser paints. useLayoutEffect runs synchronously before the paint. Use useLayoutEffect only when reading DOM measurements to prevent visual flicker.

Q Expert

Q7. Why does useEffect run twice in development?

➜ React 18 Strict Mode intentionally mounts, unmounts, and remounts components to expose missing cleanup logic. If your effect breaks on double mount, your cleanup is incomplete. Production only runs once.

Q Expert

Q8. How do you handle race conditions in useEffect?

➜ Use AbortController. Pass its signal to fetch, and call controller.abort() in the cleanup. When the dependency changes before the fetch completes, the old request is cancelled — preventing stale data from overwriting fresh data.